

Enhancing Trend-Following Strategies Using Machine Learning and Time Series Models

Spyros K. Chandrinios^a (chandrinios.spyros@gmail.com), Nikos D. Lagaros^a
(nlagaros@central.ntua.gr)

^a Institute of Structural Analysis and Antiseismic Research, School of Civil Engineering, National Technical University of Athens, 9, Heroon Polytechniou Str., Zografou Campus, Athens 15780, Greece

Corresponding author at: Institute of Structural Analysis and Antiseismic Research, School of Civil Engineering, National Technical University of Athens, 9, Heroon Polytechniou Str., Zografou Campus, Athens 15780, Greece

Spyros K. Chandrinios

Corresponding author at: Institute of Structural Analysis and Antiseismic Research, School of Civil Engineering, National Technical University of Athens, 9, Heroon Polytechniou Str., Zografou Campus, Athens 15780, Greece

Tel: 00306949310645

Email: chandrinios.spyros@gmail.com

Enhancing Trend-Following Strategies Using Machine Learning and Time Series Models

Spyros K. Chandrinou^a, Nikos D. Lagaros^a

^a*Institute of Structural Analysis and Antiseismic Research, School of Civil Engineering,
National Technical University of Athens, 9, Heroon Polytechniou Str., Zografou Campus,
Athens 15780, Greece*

Abstract

In the study, machine learning (ML)-based trends following techniques are used to include ML models in trading and investing. Trend-following strategy based on Ichimoku Cloud indicator is complemented by cutting-edge ML models such as Naive Bayes, K-Nearest Neighbors, Gradient Boosting, Decision Trees, and others. They are backtested using currency pairs (2010-2023) data, and their relative performance against parameters such as total return, maximum draw-down, and mean return. Data indicate that Extreme Gradient Boost (XGB) or Naive Bayes algorithms give far better returns at low risk than the standard approach. Additionally, time series models are also used (as Temporal Convolutional Network and Kalman filters) for accurate predictions. Both models have major upsides on total returns and mean return with lower volatility. The study describes how machine learning and time series models can be used to improve decision making in the financial markets and increase the profitability of trend-following strategies.

Keywords: ichimoku cloud, trend-following, trading strategy, machine learning, time-series models

*Spyros K. Chandrinou
Email addresses: chandrinou.spyros@gmail.com (Spyros K. Chandrinou),
nlagaros@central.ntua.gr (Nikos D. Lagaros)

1. Introduction

The Financial industry primarily in trading and investment is undergoing an infrastructural change due to machine learning (ML). With ML models entering these areas, decisions and strategies have never been so much easier to analyse data. Traditional approaches, mostly based on human intuition and historical statistics, are getting supplemented – or in many cases displaced – by advanced algorithms that are able to deal with immense amounts of information, identify patterns, and predict with faster and better precision than ever before. The ever-increasing volatility of financial markets has also made these models more widely popular, allowing traders and investors to compete by being more educated, the so called data-driven decisions. With the revolving door of the financial sector, it's important for practitioners as well researchers to know the impact of machine learning on trading and investing.

In recent years, it has become very popular to use machine learning models in trading and investing, in the trend following scenarios. Trend-following, as presented in classics such as ? paper on momentum of time series, explains that the assets with strong past returns also tend to return to a high level in the short term which could be applied to future predictive modelling of financial markets. This approach also has historical support, such as the detailed review by ? that calculates trend-following strategy profitability over time across different asset classes. ?: AIRMS is a machine learning-driven risk management decision support system — allowing prediction and management of financial risk.

It is also important to note that the mechanics behind these moves have been investigated in other studies such as ? study of the permanent and temporal components of stock prices, with consequences for price dynamics and the longevity of market moves. And then there are the feedback traders, where ? goes on to show how the movement of prices past could be the basis of the market for the future (an idea at the core of trend-following).

Managed futures, a popular market place asset which uses trend-following, and ? report a description of these strategies' risk-return profiles. The imple-

mentation guide of these techniques can be found in [1]. Providing an in-depth guide for investors who want to implement trend-following techniques effectively. [2] – they propose a currency portfolio-building system using a trend-following optimized investment strategy. This method intelligently allocates funds for highest possible return based on market fundamentals and is a structured approach to managing currency portfolios in volatile markets. These authors in their turn point to the success of trend-following in all kinds of markets, making the way for the addition of cutting edge machine learning algorithms to improve upon these tactics even more.

2. Trading Strategy

Our research interests focus on trend-following decision-making techniques, a widely used trading method for capitalizing on longer-term price movements in financial markets. The essence of trend-following is to identify and exploit upward or downward trends in asset prices. These approaches operate under the assumption that once a trend begins, it will persist for a certain period, allowing investors to leverage price directionality. Recent advancements in machine learning and time series models, as it is explored in this study, have significantly improved the predictive capabilities of trend-following strategies by refining pattern recognition and dynamic risk management. This decision-making process is especially applicable to our goal of optimizing cost efficiency in additive manufacturing (AM) within the construction industry. AM technologies, such as 3D concrete printing, rely heavily on material usage forecasting, production scheduling, and demand-supply optimization to minimize waste and enhance sustainability. By leveraging trend-following methodologies enhanced with machine learning, we can develop predictive models for material consumption trends, dynamically adjust printing parameters, and optimize procurement strategies, ultimately reducing costs and environmental impact. This approach aligns with ongoing research on AI-driven decision-making in construction (e.g., [3]), demonstrating the potential of financial market strategies in improving

industrial-scale 3D printing efficiency.

We have chosen the Ichimoku Cloud as our main trend-following indicator after conducting a lot of correlation and comparison. This multi-purpose, complete technical evaluative instrument was called Ichimoku Kinko Hyo and created by the Japanese journalist Goichi Hosoda in the late 1930s. Ichimoku Cloud gives you a full-body picture of the price direction, using various lines and parts to provide information about the market, support/resistance, entry, and exit points. The Ichimoku Cloud chart, as illustrated in Figures 1, displays the different components of the Ichimoku indicator: Tenkan-Sen, Kijun-Sen, Senkou Span A, Senkou Span B, Chiku Span, and Kumo (cloud), which collectively provide insights into trend direction, momentum, and potential reversal zones. We can simply say that *Ichimoku Kinko Hyo*, meaning *one glance equilibrium chart* in Japanese, perfectly reflects its ability to give you a complete market picture at a glance.

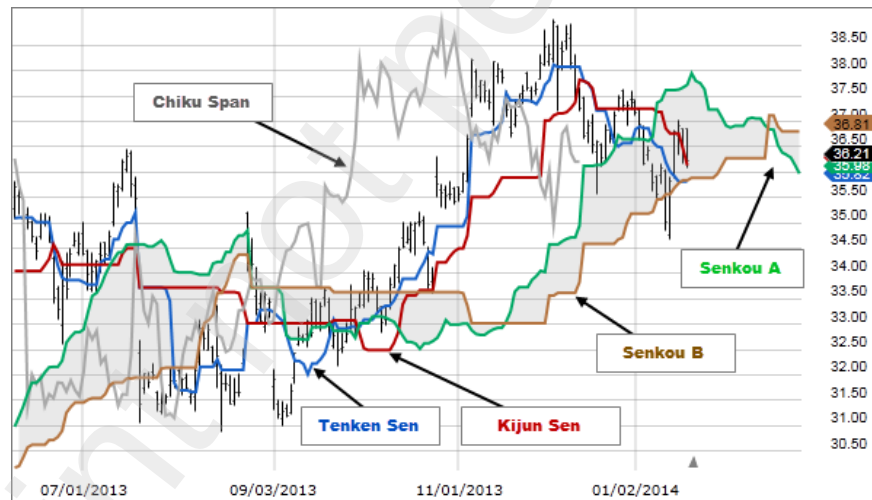


Figure 1: Ichimoku Cloud chart that shows the different components of Ichimoku indicator: Tenkan-Sen, Kijun-Sen, Senkou Span A, Senkou Span B, Chiku Span, Kumo (cloud).

The Ichimoku Cloud consists of several components, including:

- Tenkan-Sen (Conversion Line): This is the shortest line and it's the mean

of high and low in the past 9 periods. It is the short-term force of the market.

- Kijun-Sen (Base Line): This is the longer-term line, calculated similarly to Tenkan-Sen. but for the previous 26 periods. It is the market's future pattern.
- Senkou Span A (Leading Span A): In the middle of Tenkan-Sen and Kijun-Sen. plotted 26 periods ahead. It's the first edge of the cloud, as well as one of the clouds borders.
- Senkou Span B (Leading Span B): This is halfway point between the highest high and the lowest low in the last 52 periods. plotted 26 periods ahead. It is the cloud's second line and another boundary.
- Chikou Span (Lagging Span): The last price for the current period. plotted 26 dates in the past. It also allows traders to know the intensity of the trend based on historical price movements.
- Kumo (Cloud): Cloud consist of the region between Senkou Span A and Senkou Span B. The cloud acts as a live support/resistance line and can be used to help traders see whether the trend reverses or continues.

3. Application of machine learning models into the trading strategy

The use of machine learning algorithms for trading is on the rise, proving to be much more effective than old approaches. Recently, ? introduced a trading model using deep reinforcement learning and imitative learning to optimize quantitative trading strategies, framed within a Partially Observable Markov Decision Process (POMDP). ? tried deep neural networks, gradient-boosted trees, & random forests for statistical arbitrage rules on the S&P 500, demonstrating that these models are capable of significantly improving trading performance. The same goes for ? predicting stock returns using statistical

learning models including support vector machines and random forests, particularly with limited data sets, and found that such models successfully represent elaborate financial cycles.

Deep learning techniques were also shown to provide further machine learning advances. [?], who were using Long Short-Term Memory (LSTM) networks to predict stock prices, showed that the capabilities of deep learning models to identify temporal dependencies in financial data improves the accuracy of predictions and trade execution. [?] even employed convolutional neural networks for enhancing momentum trading, pointing to how deep learning can help to increase the profitability of these legacy methods.

Moreover, the application of machine learning to a dynamic trading approach, was used with reinforcement learning ([?]). They simulated investor behaviour under prospect theory to optimize consumption and portfolio selection, showing how machine learning can be used to improve financial market decision-making.

In our application of machine learning algorithms, we've mostly used them as a classification scheme to scrub generated signals to get at the ones that are most likely to make money and minimize the risk involved instead of directly boosting the overall system's performance.

3.1. Machine learning models

The machine learning models that we have used in our research are the following ones:

- The Naive Bayes Classifier is a probabilistic model rooted in Bayes' theorem, designed for classification tasks. It is particularly effective in scenarios where the data is high-dimensional and the assumption of feature independence can approximate reality. Despite its "naive" assumption of independence, it often achieves competitive performance in many practical applications. The Naive Bayes classifier calculates the posterior probability of a class C given a set of features $X = \{x_1, x_2, \dots, x_n\}$ using Bayes'

theorem. It assumes that the features are conditionally independent given the class label. The equation is expressed as:

$$P(C | X) \propto P(C) \prod_{i=1}^n P(x_i | C) \quad (1)$$

where $P(C | X)$ refers to the posterior probability of class C given the feature set X ; $P(C)$ is the prior probability of the class C , i.e., the likelihood of C occurring without considering the features; $P(x_i | C)$ is the likelihood of feature x_i given class C and $\prod_{i=1}^n$ is the product of probabilities for all features, assuming independence. The classifier predicts the class C with the highest posterior probability $P(C | X)$.

- K-Nearest Neighbors (KNN): A simple, instance-based learning algorithm that classifies data points by majority vote among the k -nearest neighbors in the feature space, making it highly interpretable but sensitive to the choice of k . Given a test point x , the algorithm finds the k closest training points in the feature space and assigns the most common label. The predicted label $\hat{y}$ for a test point x is determined as:

$$\hat{y} = \text{mode}(y_i | x_i \in N_k(x)) \quad (2)$$

where $\hat{y}$ is the predicted label for the test point x , $N_k(x)$ denotes the set of the k -nearest neighbors of x and y_i is the label of the i -th neighbor.

- Gradient Boosting: An ensemble technique that builds models sequentially, with each new model correcting errors made by the previous ones, leading to highly accurate predictions, especially in the presence of complex patterns. It minimizes a loss function $L(y, F(x))$ by fitting new models to the residual errors of previous ones. The model at iteration m is updated as:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x) \quad (3)$$

where $F_m(x)$ is the model after m iterations, $F_{m-1}(x)$ is the model from the previous iteration, η is the learning rate and $h_m(x)$ is the new model (usually a decision tree) fitted to the residual errors.

- **Decision Trees:** A non-parametric model that splits the data into branches based on feature values. creating a tree-like structure that is easy to interpret and useful for both classification and regression tasks. The tree is built by recursively partitioning the feature space, aiming to minimize a loss function (such as Gini impurity or mean squared error). The recursive splitting can be represented as:

$$\text{Split Criterion} = \underset{t}{\operatorname{argmin}} \sum_{i=1}^n (\text{Loss}(y_i, f_t(x_i))) \quad (4)$$

where t represents the split threshold, y_i are the target values, $f_t(x_i)$ is the prediction for input x_i at split t and the loss function measures the error or impurity after splitting.

- **Random Forest:** An ensemble method that combines multiple decision trees, each trained on a random subset of the data to improve accuracy and reduce overfitting by averaging the predictions. The final prediction is made by aggregating the outputs from all individual trees:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(x) \quad (5)$$

where $\hat{y}$ is the final predicted value, T is the number of trees and $f_t(x)$ is the prediction from the t -th tree for input x .

- K-Means Clustering: An unsupervised learning algorithm that partitions data into k clusters by minimizing the variance within each cluster, making it effective for identifying underlying patterns in unlabeled data. The algorithm iterates between assigning each data point to the nearest centroid and updating the centroids based on the mean of assigned points. The objective function is given by:

$$J = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2 \quad (6)$$

where J is the total within-cluster variance, C_i is the set of points in cluster i , x_j is a data point in cluster C_i , μ_i is the centroid of cluster i and $\|x_j - \mu_i\|^2$ is the squared Euclidean distance between point x_j and centroid μ_i .

- Gaussian Mixture Models (GMM): A probabilistic model that represents the data as a mixture of several Gaussian distributions, allowing for the flexible modeling of data with underlying subpopulations. Each Gaussian component is characterized by a mean vector and covariance matrix. The likelihood of a data point x given the model is:

$$p(x|\Theta) = \sum_{i=1}^k \pi_i \mathcal{N}(x|\mu_i, \Sigma_i) \quad (7)$$

where $p(x|\Theta)$ is the probability density of the data point x , k is the number of Gaussian components, π_i is the weight of the i -th component (with $\sum_{i=1}^k \pi_i = 1$) and $\mathcal{N}(x|\mu_i, \Sigma_i)$ is the Gaussian distribution with mean μ_i and covariance Σ_i for the i -th component.

- Extreme Gradient Boost (XGBoost): An advanced implementation of gradient boosting that uses optimized algorithms for better performance and speed, making it particularly effective for large-scale and complex datasets.

XGBoost enhances the traditional gradient boosting approach by incorporating regularization (L1 and L2) to prevent overfitting and implementing more efficient tree-building algorithms. The objective function in XGBoost is:

$$\mathcal{L}(\Theta) = \sum_{i=1}^n \mathcal{L}(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (8)$$

where $\mathcal{L}(y_i, \hat{y}_i)$ is the loss function measuring the difference between the predicted $\hat{y}_i$ and the true label y_i and $\Omega(f_k)$ is the regularization term for the k -th tree, typically defined as:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}_k\|^2 \quad (9)$$

where T is the number of leaves in the tree, γ and λ are the regularization parameters and $\mathbf{w}_k$ represents the weights of the tree.

XGBoost uses this objective function to balance fitting the data and regularizing the model, improving both model accuracy and generalization.

3.2. Back-testing results

In our study, we have utilized extensive currency pair data due to its microstructural similarities with commodity markets, such as oil, steel, and copper. These materials play a crucial role in the construction sector, particularly in additive manufacturing, where precise material cost forecasting and procurement strategies are essential. Studies such as ? and ? highlight the interdependence of commodity markets and construction material pricing, reinforcing the necessity for predictive financial models in managing AM supply chains. To validate our trend-following decision-making strategy, we conducted comprehensive back-testing from 2010 to 2023, employing advanced machine learning models to assess predictive accuracy. Below, we present the performance results, including total return, maximum drawdown, mean return, standard deviation, and the number of trades. Our findings highlight the robustness of AI-enhanced

trend-following techniques in identifying optimal decision points for resource allocation in both financial and industrial domains, reinforcing the applicability of financial forecasting methodologies in optimizing additive manufacturing workflows.

Table 1: Total returns (%) from backtesting period 2010-2023 by applying machine learning models

Strategy	EUR/USD	USD/JPY	AUD/USD	GBP/USD	NZD/USD
Strategy	71.64	29.00	40.92	75.92	42.18
XGB	97.22	36.79	45.51	82.06	55.89
GMM	26.13	62.04	21.28	66.35	50.96
K-Means	64.81	28.67	38.87	31.28	26.59
RF	70.39	17.92	22.94	35.57	57.11
DT	38.02	48.54	10.98	28.81	37.51
GBM	55.36	15.44	12.21	63.96	37.56
kNN	41.79	14.99	33.00	49.27	19.93
NB	88.62	43.26	49.92	96.56	62.64

Note: The table presents the total returns (%) obtained from backtesting various machine learning models from 2010 to 2023 across different currency pairs.

Table 2: Number of trades from backtesting period 2010-2023 by applying machine learning models

Strategy	EUR/USD	USD/JPY	AUD/USD	GBP/USD	NZD/USD
Strategy	1986	2070	2037	2108	2050
XGB	972	1009	913	1076	962
GMM	1149	1155	1153	1263	1301
K-Means	1192	1172	1236	1295	1215
RF	949	931	936	1005	958
DT	995	1047	899	1041	965
GBM	951	939	909	1027	981
kNN	702	1006	692	786	688
NB	1187	1138	1238	1307	1302

Note: The table reports the total number of trades executed during the backtesting period (2010-2023) for different machine learning models across various currency pairs.

Table 3: Maximum drawdown (%) from backtesting period 2010-2023 by applying machine learning models

Strategy	EUR/USD	USD/JPY	AUD/USD	GBP/USD	NZD/USD
Strategy	16.21	25.91	15.46	12.08	11.18
XGB	6.56	16.03	12.41	9.34	7.03
GMM	20.56	15.42	13.74	14.16	8.70
K-Means	10.19	21.71	18.60	17.19	16.50
RF	8.54	26.84	16.71	9.53	7.89
DT	15.31	20.41	16.65	13.96	7.51
GBM	7.50	23.45	16.92	7.18	10.02
kNN	10.72	25.22	12.61	13.26	13.71
NB	9.44	17.53	14.76	9.98	9.60

Note: The table shows the maximum drawdown (%) for each machine learning model during the backtesting period (2010-2023) for different currency pairs.

Table 4: Mean return (%) from backtesting period 2010-2023 by applying machine learning models

Strategy	EUR/USD	USD/JPY	AUD/USD	GBP/USD	NZD/USD
Strategy	0.037	0.015	0.020	0.039	0.021
XGB	0.100	0.037	0.050	0.076	0.058
GMM	0.022	0.054	0.018	0.052	0.039
K-Means	0.054	0.024	0.032	0.024	0.022
RF	0.074	0.019	0.025	0.035	0.059
DT	0.038	0.046	0.012	0.028	0.038
GBM	0.057	0.016	0.013	0.062	0.038
kNN	0.059	0.015	0.047	0.062	0.027
NB	0.075	0.038	0.040	0.074	0.048

Note: The table presents the mean return (%) for each machine learning model during the backtesting period (2010-2023) for various currency pairs.

Table 5: Standard deviation (%) from backtesting period 2010-2023 by applying machine learning models

Strategy	EUR/USD	USD/JPY	AUD/USD	GBP/USD	NZD/USD
Strategy	0.78	0.81	0.73	0.78	0.70
XGB	0.73	0.78	0.68	0.74	0.66
GMM	0.75	0.90	0.78	0.78	0.75
K-Means	0.74	0.91	0.79	0.76	0.73
RF	0.73	0.80	0.78	0.80	0.72
DT	0.76	0.92	0.75	0.77	0.73
GBM	0.76	0.85	0.75	0.78	0.73
kNN	0.80	0.83	0.83	0.94	0.75
NB	0.73	0.81	0.73	0.74	0.69

Note: The table displays the standard deviation (%) for each machine learning model during the backtesting period (2010-2023) across different currency pairs.

The results presented above demonstrate that the application of the Extreme Gradient Boost (XGB) model leads to an increase in total returns across all assets when compared to the benchmark basic strategy as shown in Table 1. A similar improvement is observed with the Naive Bayes (NB) classifier model. Additionally, in Table 3, both XGB and NB models exhibit lower maximum drawdowns across all assets relative to the original strategy, indicating a reduction in risk. However, the k-Nearest Neighbors (kNN) model consistently underperforms the original Ichimoku strategy across all currency pairs. As we observe from the above tables, the remaining models do not show consistent behavior across all assets.

Another key finding is that models such as XGB, Random Forest (RF), Decision Trees (DT), Gradient Boosting Machines (GBM) and kNN significantly reduce the number of trades as shown in Table 2, which is a positive outcome in terms of lowering trading costs. Despite kNN generating fewer trades and producing lower total returns than the original strategy, it yields a higher mean return (Table 4), likely due to the reduced trading frequency.

4. Application of time series models into the trading strategy

The prediction of financial time series has long been a central challenge in the field of quantitative finance, given the inherent complexity, volatility, and non-stationarity of financial markets. Among the various approaches to financial forecasting, time series models have emerged as a fundamental tool, offering powerful methods for analyzing and predicting future price movements based on historical data. The time series models are primarily built to extract temporal dependences and structure of sequential financial data. Standardised models (like autoregressive integrated moving average (ARIMA) or exponential smoothing) have been widely applied because they are easy to interpret. But as high-frequency data became more available and computation power grew, newer models have emerged, such as state-space models, Generalized Autoregressive Conditional Heteroskedasticity (GARCH) models and more advanced machine

learning techniques such as Long Short-Term Memory (LSTM) networks, Temporal Convolutional Network (TCN) and other deep learning systems.

Financial indexes have also been predicted using multivariate time series models, such as ?, who applied the models to build trading strategies based on forecasted index moves. Hybrids that combine traditional models with new techniques, like the ARIMA/ neural network combination that ? has developed, have further improved prediction accuracy for the financial markets. ? provides the best of the latest trading strategy. ? has also been influential, with his GARCH model, a powerful tool to simulate financial time series with a fluctuating volatility that helps us to predict risk and volatility in the markets. ? is the leading work on financial time series and describes the use of models such as GARCH to estimate stock prices, volatility, and returns and it is a must-read for understanding and using these techniques in financial forecasting.

We didn't just categorize signals, we added time series models to the Ichimoku Cloud strategy to increase the precision of its original signals. With time series analysis and forecasting by the Python library *DARTS*, we tried to model the Ichimoku components to have a better idea of their near-future trajectory. In this way, we are able to build a more accurate system with fewer false signals. In particular, we used two models for modeling the Ichimoku pieces, namely the TCN model and the Kalman Forecaster.

4.1. Temporal Convolutional Network (TCN) Model

? give an overall empirical review of TCNs as compared to recurrent networks for sequence modeler tasks. with emphasis on the performance and cost-efficiency gains of TCNs. which is an argument in favour of them for time series prediction. ? first introduced TCNs in a more general manner where they have shown their ability to represent sequential data and apply that directly to time series models in finance. Another example by ? is a joint application of TCNs and LSTM networks to stock market forecasting: TCNs identify temporal dependences in financial data.

Temporal Convolutional Network (TCN) is a deep learning architecture for

sequence modelling like time series prediction. It also well combats shortcomings of traditional RNNs and LSTM networks (vanishing gradients and slow training rates). The long-range dependencies in time series data can be identified using TCNs that use dilated causal convolutions and are without need of recurrent layers. This is the capability of TCNs to model non-linear and non-linear relationships in financial time series that traditional models such as ARIMA or GARCH are not able to handle. By inferring the behavior from the past, TCNs can also be used in trading applications to predict future price movements and volatility with a stronger and more adaptive prediction framework. Trading strategies can include TCNs to generate more accurate prediction, which will ultimately result in better informed trading decisions and higher performance across various financial assets.

The basic elements of the TCN architecture are:

1. **1D Convolutional Layers:** These are 1D convolutions with specified filters, kernel sizes, and dilation rates. The convolutions are causal, meaning they do not take future data into account when making predictions.
2. **ReLU Activation:** The ReLU (Rectified Linear Unit) activation function introduces non-linearity into the model, enabling it to learn complex patterns.
3. **Normalization:** Weight normalization or batch normalization is employed to stabilize and accelerate the training process.
4. **Dropout:** Dropout is used to prevent overfitting by randomly setting a fraction of the input units to zero during training.
5. **Residual Blocks:** Each convolutional layer is part of a residual block that passes the block input directly to the output. This helps preserve the gradient during backpropagation, ensuring efficient training of deep networks.

To mathematically express the core concept of the Temporal Convolutional Network (TCN), we represent the convolution operation within a residual block,

emphasizing the dilation factor, which is critical for capturing long-range dependencies in sequence data.

The output of the convolution at time t , denoted y_t , is expressed as:

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-d \cdot i} + b \quad (10)$$

where w_i are the learnable weights associated with the convolution filter, d is the dilation factor, which expands the receptive field by skipping values of x_t based on the dilation rate (this allows TCNs to model dependencies over longer temporal spans without excessively increasing the depth of the network), k is the kernel size, representing the number of data points considered in the convolution operation, b is the bias term added to the convolution output, improving the model's capacity to fit data and $x_{t-d \cdot i}$ represents the input value at a lag of $d \cdot i$, ensuring causality by considering only past values up to the current time step t .

Key properties of this formulation include:

- **Causality:** By using only past values ($t - d \cdot i$), the convolution ensures that predictions at time t do not incorporate future data, maintaining the temporal order.
- **Receptive Field Expansion:** The dilation factor d exponentially increases the receptive field of the convolution with respect to the depth of the network, allowing TCNs to capture patterns over long sequences efficiently.
- **Residual Connections:** The residual block structure incorporates a skip connection that passes the input x_t directly to the output, aiding gradient flow during backpropagation and mitigating the vanishing gradient problem.
- **Computational Efficiency:** Unlike recurrent models, TCNs allow parallel computation during training and inference by processing the entire input sequence at once.

This formulation underpins the TCN's ability to model sequential and time-series data effectively, leveraging its unique architecture to address challenges like vanishing gradients, slow training rates, and the need for long-range dependency modeling.

4.2. Kalman Forecaster

In his pioneering article, [1], he developed the Kalman filter that is now being used extensively in many different disciplines such as finance. [1] gives a clear and extensive discussion of Kalman filter's contribution to the predictive and structural time series modelling, providing valuable guidance for its use in financial markets. [2] describe how the Kalman filter is applied to state-space models in regime-switching settings, a feature extremely relevant to financial time series analysis. [3] provide an exhaustive treatment of state-space models and the Kalman filter in the context of financial time series analysis, such as asset price models and risk management. [4] take the classic Kalman filter theory to a whole new level, bringing theoretical innovations that have practical applications in financial markets such as algorithmic trading and portfolio management. In addition, [5] discuss the Kalman filter in the context of feedback control systems (which applies to financial trading systems that need to be continuously recalculated and optimized).

The Kalman Forecaster is based on the Kalman Filter, an advanced state-space model known for forecasting the state of a dynamic system from a stream of noisy measurements. The Kalman Filter is a powerful algorithm used in control systems, signal processing and economics that excels at recursively estimating process states (i.e., the actual price of an asset) and removing noise from the data.

The Kalman Filter is a critical part of time series analysis and trading strategies in the financial industry. Its capacity to deal with uncertainty and volatility also lends itself perfectly well in situations where the data is noisy and highly dynamic. As the filter is recursive, its estimations are constantly being updated with new information, which is really useful when dynamically tuning model

parameters, detecting patterns, and performing predictive tasks in real-time. This feature is very helpful when tracking moving averages and applying mean-reversion strategies. We find that Kalman Filter's process of improving the prediction over time helps traders to be more dynamic and responsive, making it more accurate to make predictions and make trading more efficient in highly volatile markets.

The Kalman Filter is an optimal recursive algorithm used for estimating the state of a dynamic system by minimizing the mean squared error. Its primary application in finance is to filter out noise and handle real-time updates, making it valuable for trading strategies and risk management.

The Kalman Filter operates in two main steps:

1. **Prediction:** Forecast the next state based on system dynamics.
2. **Update:** Refine the forecast using new observations.

Let the system state at time t be $\mathbf{x}_t$, and let $\mathbf{z}_t$ represent the observed measurements. The Kalman Filter is defined by the following equations:

State Prediction

$$\hat{\mathbf{x}}_t^- = \mathbf{F}\hat{\mathbf{x}}_{t-1} + \mathbf{B}\mathbf{u}_t, \quad (11)$$

$$\mathbf{P}_t^- = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^\top + \mathbf{Q}. \quad (12)$$

Measurement Update

$$\mathbf{K}_t = \mathbf{P}_t^- \mathbf{H}^\top (\mathbf{H}\mathbf{P}_t^- \mathbf{H}^\top + \mathbf{R})^{-1}, \quad (13)$$

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_t^- + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_t^-), \quad (14)$$

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_t^-. \quad (15)$$

Variables and Notation

- $\hat{\mathbf{x}}_t^-$: Predicted state estimate at time t .
- $\hat{\mathbf{x}}_t$: Updated state estimate at time t .
- $\mathbf{P}_t^-$: Predicted error covariance matrix at time t .

- $\mathbf{P}_t$: Updated error covariance matrix at time t .
- $\mathbf{F}$: State transition matrix.
- $\mathbf{B}$: Control input matrix.
- $\mathbf{u}_t$: Control input at time t .
- $\mathbf{H}$: Observation model matrix.
- $\mathbf{Q}$: Process noise covariance matrix.
- $\mathbf{R}$: Measurement noise covariance matrix.
- $\mathbf{K}_t$: Kalman gain matrix.
- $\mathbf{z}_t$: Observed measurement at time t .
- $\mathbf{I}$: Identity matrix.

The Kalman Filter's ability to dynamically adapt its predictions based on incoming data makes it particularly suited for applications such as:

- Tracking moving averages and mean-reversion strategies in trading.
- Regime-switching models to identify market conditions.
- Portfolio optimization and real-time risk management.

By leveraging its recursive structure, the Kalman Filter provides a robust framework for handling noisy and volatile financial data, resulting in improved predictive accuracy and better decision-making.

4.3. Back-testing results

Table 6: Total returns (%) from back-testing period 2010-2023 by applying time series models

Strategy	EURUSD	USDJPY	AUDUSD	GBPUSD	NZDUSD
Strategy	71.64	29.00	40.92	75.92	42.18
TCN	153.99	117.71	145.71	116.79	139.24
Kalman Filter	169.28	119.84	183.67	158.34	155.98

Note: The table shows the total returns (%) from the back-testing period (2010-2023) using different time series models for various currency pairs.

Table 7: Standard deviation (%) from back-testing period 2010-2023 by applying time series models

Strategy	EURUSD	USDJPY	AUDUSD	GBPUSD	NZDUSD
Strategy	0.78	0.81	0.73	0.78	0.70
TCN	0.77	0.82	0.729	0.73	0.70
Kalman Filter	0.76	0.82	0.731	0.78	0.70

Note: The table presents the standard deviation (%) from the back-testing period (2010-2023) using time series models across different currency pairs.

Table 8: Mean return (%) from back-testing period 2010-2023 by applying time series models

Strategy	EURUSD	USDJPY	AUDUSD	GBPUSD	NZDUSD
Strategy	0.037	0.015	0.02	0.039	0.021
TCN	0.078	0.06	0.07	0.059	0.071
Kalman Filter	0.085	0.06	0.09	0.079	0.078

Note: The table presents the mean return (%) from the back-testing period (2010-2023) using time series models across various currency pairs.

Table 9: Maximum drawdown (%) from back-testing period 2010-2023 by applying time series models

Strategy	EURUSD	USDJPY	AUDUSD	GBPUSD	NZDUSD
Strategy	23.84	31.6	20.95	16.80	16.92
TCN	14.57	16.25	15.87	14.62	11.87
Kalman Filter	13.62	19.96	12.70	22.75	12.6

Note: The table presents the maximum drawdown (%) from the back-testing period (2010-2023) using time series models across various currency pairs.

It is clearly evident from these results that the introduction of time series models into the original approach improves its performance dramatically. In Table 8, the total returns of the strategies based on time series models consistently outperform the original strategy across all assets. Among these, the Kalman filter-based strategy yields higher total returns than the Temporal Convolutional Network (TCN) strategy in all currency pairs. However, in Table 7, regarding the standard deviation of returns, time series models do not introduce any meaningful differences. Another noteworthy finding is that the time series-based strategies effectively reduce the maximum drawdown as we can see in Table 9. This, combined with higher total returns, makes the TCN and Kalman filter strategies considerably more effective in terms of risk-return performance compared to the original strategy.

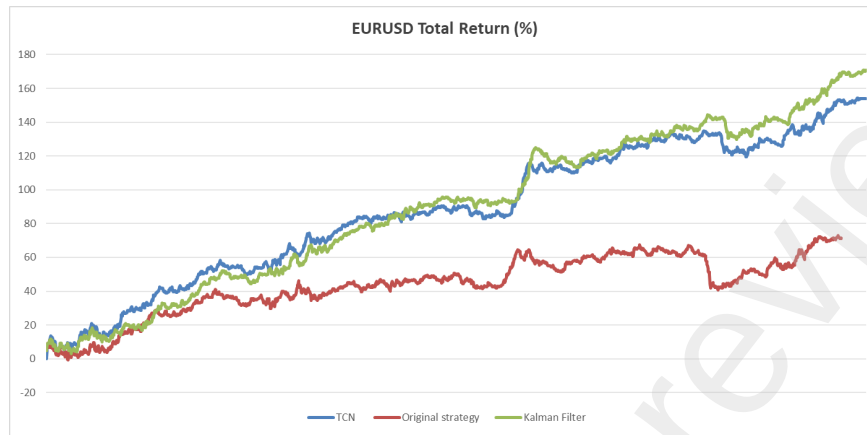


Figure 2: EURUSD: Total return graph (%) from back-testing period 2010-2023 showing the performance of the original strategy and the effect of applying TCN and Kalmanfilter models.

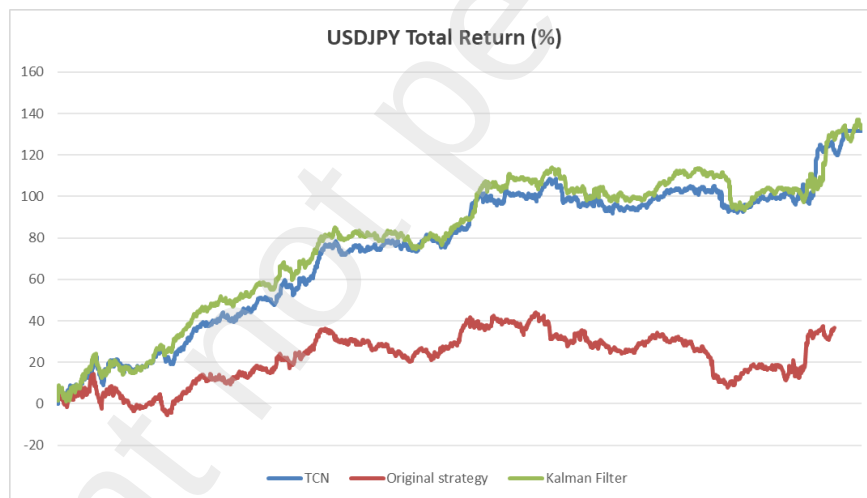


Figure 3: USDJPY: Total return graph (%) from back-testing period 2010-2023 showing the performance of the original strategy and the effect of applying TCN and Kalmanfilter models.

5. Conclusion

This study explored the integration of machine learning models with a trend-following trading strategy. Specifically using the Ichimoku Cloud indicator to enhance decision-making in predicting market trends. By leveraging the Ichimoku Cloud's comprehensive view of market dynamics, we aimed to filter and optimize trading signals with advanced ML techniques to improve accuracy and profitability in trend-following strategies. Our result shows that Machine learning models and especially ensemble approaches like Gradient Boosting, Random Forest can be applied in the right direction to improve classic trend-following approaches. These models outperformed less complicated classifiers such as Naive Bayes and K-Nearest Neighbors in the recognition of profitable signals and showed that they can identify the subtleties of financial markets. Specifically, Gradient Boosting and Extreme Gradient Boost (XGBoost) always gave better prediction performance and reduced risk over Ichimoku generated signals. Furthermore, we test the theory that when you combine machine learning with technical signals like the Ichimoku Cloud, you get better and more profitable trading strategies, Machine learning models can predict and target trading signals with the least number of false positives which makes it very superior to using just rule based models. The combination of classic financial theory and contemporary computation techniques in this hybrid solution is effective for trend-following in highly volatile and ambiguous markets.

Forecasts for movements in the financial markets have been a success using time series models. with some real advantages over standard technical analysis. We will see how using the newer time series models (TCN, Kalman Forecaster etc.) in trading algorithms with Ichimoku Cloud indicator works well. The results demonstrate emphatically that combining these models significantly improves the predictive and risk-adjusted performance of trend-following methods. In particular, TCN model which accounts for long-range dependencies and non-linear relationships in financial time series improves total returns and drawdown with substantial reduction in multiple currency pairs.

The Kalman Forecaster, instead, uses recursive filtering to adjust forecasts as data change which is especially useful in volatile markets. Kalman Filter Strategy Outperformed TCN on all pairs of currency with better total returns and R/R ratios. Furthermore, because it kept on updating its estimates of market direction and volatility, it was able to take smarter, more responsive trades. Machine Learning models (XGBoost, Naive Bayes and other) are good at identifying the trades that can be profitable, but time series like TCN and Kalman Forecaster provides high total returns and lower risk. Machine learning models can filter signals better and make the trade cheaper but time series models can better identify trends and volatility in time and hence are more appropriate for long-term trading.

The research could be built upon in the future using deep learning models like Long Short-Term Memory (LSTM) networks or CNNs to achieve still better signal classification and model efficiency. Furthermore, if this hybrid method could be applied to other asset classes and market environments, then there would be a greater picture on the generalizability and scalability of these strategies. All in all, machine learning and time series models applied to the classical trading strategies holds enormous promise to improve the algorithmic trading space and give traders better tools to leverage on a growing array of increasingly complex financial markets.

Acknowledgements

The support of the Hellenic Foundation for Research and Innovation (H.F.R.I.) under the “2nd Call for H.F.R.I. Research Projects to support Faculty Members and Researchers”, AMOSS project: “Additively Manufactured Optimized 3D Printed Steel Structures”, (Project Number: 02779) is highly acknowledged.